# Trabalho 1 – Concorrência e Sincronização: O Problema dos Filósofos Jantando

**Disciplina:** FPPD · **Grupo:** Henrique Horch, Felipe Araujo, Eduardo Costa · **Repositório:** https://github.com/eduardocsilveira/FPPD_T1

## 1. Soluções e mecanismos de sincronização

As quatro versões foram escritas em Go, cada uma em um arquivo (`v0/main.go` a `v3/main.go`). Todas usam os mesmos parâmetros: 5 filósofos, 20 ciclos, 50 ms para pensar e 50 ms para comer. A única coisa que muda entre elas é a forma de adquirir os garfos. O filósofo *i* usa o garfo *i* à esquerda e o garfo (*i*+1) mod 5 à direita.

| Versão | Como funciona | Mecanismo em Go |
|---|---|---|
| **V0** | Dois laços: para cada filósofo, 20 ciclos de pensar e comer. Um filósofo só começa quando o anterior terminou. | Nenhum. Não há goroutines, então não há disputa por garfo. |
| **V1** | Uma goroutine por filósofo. Todos pegam o garfo esquerdo, esperam 5 ms e pegam o direito. Cada passo é impresso no terminal. | Um `sync.Mutex` por garfo (`Lock` para pegar, `Unlock` para soltar) e `sync.WaitGroup` para aguardar as goroutines. |
| **V2** | Igual à V1, mas o filósofo só pega os garfos e come depois de travar a mesa inteira, e só destrava ao soltar os garfos. | Um `sync.Mutex` global (`mesa`), além dos mutexes dos garfos. A trava global funciona como um semáforo binário em volta de toda a refeição. |
| **V3** | Sem trava global. Cada filósofo pega primeiro o garfo de menor número e depois o de maior. Na prática só o filósofo 4 muda: ele pega o garfo 0 antes do garfo 4. | Um `sync.Mutex` por garfo, adquiridos sempre em ordem crescente de número (ordenação dos recursos). |

**Por que a V1 trava.** Os cinco filósofos terminam de pensar quase juntos e cada um pega o garfo esquerdo. Quando tentam o direito, ele já está na mão do vizinho. As quatro condições de deadlock estão presentes: **exclusão mútua** (um mutex só tem um dono), **segura e espera** (o filósofo mantém o garfo esquerdo enquanto aguarda o direito), **ausência de preempção** (ninguém tira o garfo de quem o segura; só o dono chama `Unlock`) e **espera circular** (F0 espera F1, que espera F2, e assim até F4, que espera F0).

**Por que V2 e V3 não travam.** Na V2 só um filósofo por vez tenta pegar garfos, então eles estão sempre livres: não existe segura e espera sobre os garfos. Na V3 a ordem global elimina a espera circular: quem espera um garfo só segura garfos de número menor, de modo que a cadeia de esperas sobe nos números e não consegue voltar ao início.

## 2. Análise comparativa

A V0 levou em média 10,19 s, o esperado para 100 ciclos de 100 ms executados em fila. A V2 levou 5,15 s, 1,98 vezes mais rápida que a V0. O ganho vem só de sobrepor o pensar de um filósofo com o comer de outro, porque as 100 refeições continuam em série (100 × 50 ms = 5 s). A V3 levou 2,74 s: 3,71 vezes mais rápida que a V0 e 1,88 vezes mais rápida que a V2. Com cinco garfos no máximo dois filósofos comem juntos, o que dá um piso de 2,5 s para as 100 refeições; a V3 ficou 10% acima desse piso.

O custo da sincronização excessiva aparece na espera. Na V2 cada filósofo esperou em média 150,0 ms por ciclo, contra 24,8 ms na V3, cerca de 6,1 vezes mais. O valor da V2 tem explicação direta: com cinco filósofos se revezando em uma única vaga, cada um come uma vez a cada 250 ms; tirando 50 ms de pensar e 50 ms de comer, sobram 150 ms parado na fila. A maior espera observada foi 204,7 ms na V2 e 153,1 ms na V3. O pico de simultaneidade foi 1 na V2 e 2 na V3. Na V0 não há espera, porque nunca existe outro filósofo ativo. Todos os filósofos concluíram os 20 ciclos nas três versões que terminam.

A V1 foi analisada à parte. Travou em 5 de 5 execuções, logo depois do primeiro pensar e sem nenhum ciclo concluído. O deadlock não é garantido pelo código, pois depende de os cinco pegarem o garfo esquerdo antes de alguém pegar o direito, mas é muito provável, porque todos terminam de pensar praticamente no mesmo instante.

## 3. Resultados experimentais

Médias de 5 execuções por versão. Tempos em milissegundos. A maior espera é o máximo entre todas as execuções.

| Versão | Sincronização | Tempo total (média) | Tempo total (mín – máx) | Espera média | Maior espera | Máx. comendo juntos | Ciclos por filósofo |
|---|---|---|---|---|---|---|---|
| **V0** | nenhuma (sequencial) | 10.185,5 | 10.182,1 – 10.188,8 | 0 | 0 | 1 | 20 cada |
| **V2** | trava única da mesa | 5.146,0 | 5.140,6 – 5.151,4 | 149,99 | 204,74 | 1 | 20 cada |
| **V3** | ordenação dos garfos | 2.741,8 | 2.702,5 – 2.752,6 | 24,77 | 153,11 | 2 | 20 cada |
| **V1** | mutex por garfo, esq. → dir. | | | | | deadlock em 5 de 5 execuções; 0 ciclos concluídos | |

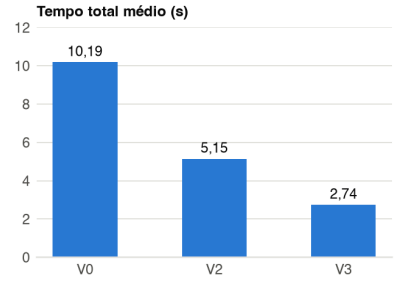
Na V0 a espera é zero e só um filósofo come por vez por construção: o programa é sequencial.

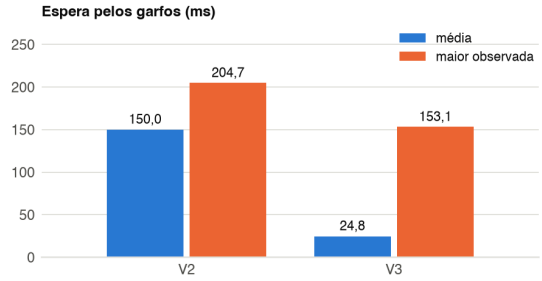
**V2, execução por execução**

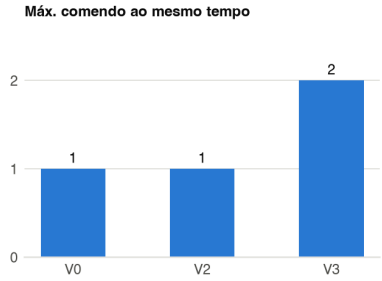
| Exec. | Tempo total | Espera média | Maior espera | Máx. juntos |
|---|---|---|---|---|
| 1 | 5.145,6 | 150,28 | 203,74 | 1 |
| 2 | 5.140,6 | 150,15 | 204,13 | 1 |
| 3 | 5.148,0 | 149,32 | 204,74 | 1 |
| 4 | 5.151,4 | 149,95 | 204,50 | 1 |
| 5 | 5.144,5 | 150,24 | 203,88 | 1 |

**V3, execução por execução**

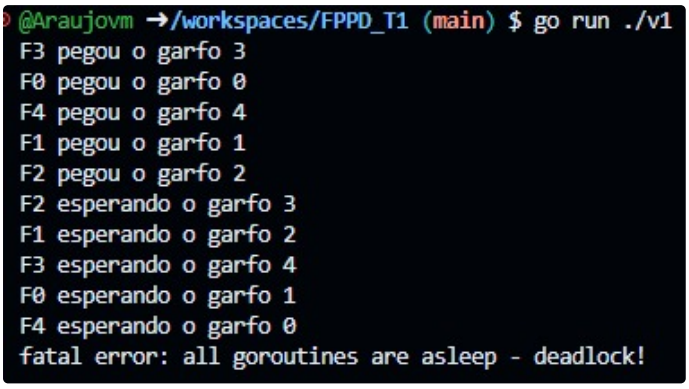
| Exec. | Tempo total | Espera média | Maior espera | Máx. juntos |
|---|---|---|---|---|
| 1 | 2.752,6 | 26,50 | 152,64 | 2 |
| 2 | 2.752,3 | 23,45 | 102,57 | 2 |
| 3 | 2.751,6 | 24,46 | 153,11 | 2 |
| 4 | 2.749,9 | 25,47 | 102,59 | 2 |
| 5 | 2.702,5 | 23,98 | 102,44 | 2 |



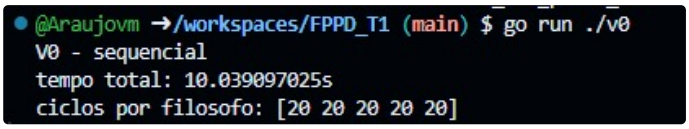



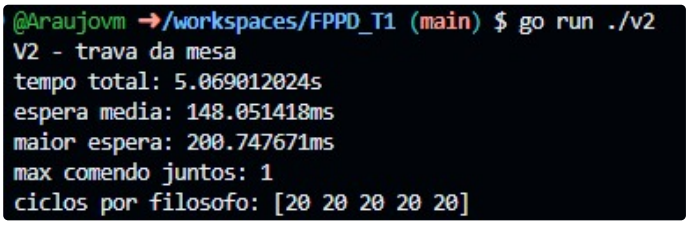



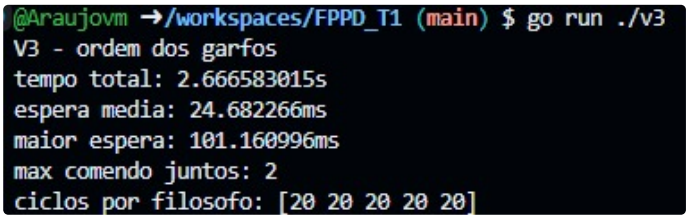

**Capturas de tela das execuções**




V1: evidência do deadlock




As capturas foram feitas no GitHub Codespaces (Linux), a partir do repositório do grupo. As medições da tabela e dos gráficos foram feitas no Apple M1 descrito na seção 5; por isso os tempos das capturas diferem um pouco dos da tabela. Na captura da V1, as linhas "pegou" mostram o que cada filósofo possui e as linhas "esperando" mostram o que cada um aguarda; como nenhuma goroutine pode ser acordada, o Go encerra o programa com `fatal error`. A saída completa de cada execução medida está em `resultados/saidas/`.

## 4. Conclusão e respostas

**1. Como funciona cada solução?**

A V0 executa os filósofos em fila, sem concorrência. A V1 cria uma goroutine por filósofo, que pega o garfo esquerdo e depois o direito. A V2 faz o mesmo dentro de uma trava que cobre a mesa inteira. A V3 dispensa a trava global e faz todos pegarem os garfos em ordem crescente de número.

**2. Quais mecanismos de sincronização são utilizados?**

Exclusão mútua por garfo na V1 e na V3, exclusão mútua global na V2 e ordenação dos recursos na V3. A V0 não usa nenhum.

**3. Como esses mecanismos são implementados em Go?**

Cada garfo é um `sync.Mutex` dentro de um vetor `[5]sync.Mutex`; `Lock` bloqueia a goroutine até o garfo estar livre e `Unlock` o devolve. A mesa da V2 é mais um `sync.Mutex`. Os filósofos são goroutines criadas com `go`, e a função principal espera por elas com `sync.WaitGroup`. A ordenação da V3 não precisa de nenhum tipo novo: é só comparar os dois números e travar o menor primeiro. As métricas ficam em variáveis globais protegidas por um mutex próprio (`mu`), que não participa da disputa pelos garfos.

**4. Por que ocorre deadlock na V1?**

Porque os cinco filósofos conseguem pegar o garfo esquerdo antes que qualquer um pegue o direito. A saída do programa (página 2) mostra o estado: F0 segura o garfo 0 e espera o 1, F1 segura o 1 e espera o 2, F2 segura o 2 e espera o 3, F3 segura o 3 e espera o 4, F4 segura o 4 e espera o 0. Cada garfo esperado está na mão do vizinho, que também está bloqueado, e o ciclo se fecha em F0. As quatro condições estão presentes ao mesmo tempo. O bloqueio ocorreu em 5 das 5 execuções, e o Go encerrou o programa com `fatal error: all goroutines are asleep - deadlock!`.

**5. Como V2 e V3 evitam deadlock?**

A V2 elimina a condição de segura e espera sobre os garfos: só quem tem a mesa tenta pegá-los, e nesse momento os dois estão livres. A V3 elimina a espera circular: com a ordem global, o filósofo 4 disputa o garfo 0 com o filósofo 0 antes de segurar qualquer coisa, então nunca existem cinco filósofos segurando um garfo cada. As duas concluíram os 100 ciclos em todas as execuções e também passaram pelo detector de corrida do Go (`go run -race`) sem avisos.

**6. Existe possibilidade de starvation?**

Na V1 travada, sim e para todos: ninguém volta a comer. Na V2 e na V3 o algoritmo não garante ordem de atendimento, porque o `sync.Mutex` não é uma fila estrita; em teoria um filósofo poderia ficar para trás se os outros sempre passassem na frente. Nos experimentos isso não ocorreu: todos os filósofos concluíram os 20 ciclos em todas as execuções, e a maior espera foi limitada, 205 ms na V2 (cerca de quatro refeições dos outros) e 153 ms na V3 (cerca de 3 refeições de vizinhos). Starvation é possível em teoria, mas não foi observada.

**7. Qual solução apresentou maior tempo total de execução?**

A V0, com 10.185 ms em média. Entre as concorrentes que concluem, a V2, com 5.146 ms, contra 2.742 ms da V3.

**8. Qual solução apresentou maior tempo de espera pelos recursos?**

A V2: 150,0 ms de espera média e 204,7 ms de maior espera. Na V3 foram 24,8 ms e 153,1 ms. Na V0 não há espera, porque nunca há outro filósofo ativo.

**9. Quantos filósofos conseguiram comer simultaneamente em cada solução?**

V0: 1. V2: 1. V3: 2, que é o máximo possível com cinco garfos. V1: 0, pois travou antes da primeira refeição.

**10. Qual foi o impacto da sincronização excessiva da V2?**

A trava global transformou as refeições em uma fila única. O tempo total ficou em 5.146 ms, praticamente os 5.000 ms das 100 refeições em série, e 1,88 vezes o tempo da V3. Cada filósofo passou em média 150 ms por ciclo esperando, três vezes o tempo que passa comendo. As cinco execuções da V2 ficaram entre 5.141 e 5.151 ms, quase sem variação, o que mostra que esse custo é estrutural e não efeito de escalonamento.

**11. A solução com maior sincronização foi necessariamente a de melhor desempenho?**

Não. A V2 é a que mais restringe os filósofos e foi a pior das concorrentes em todas as métricas: 1,88 vezes o tempo total da V3, 6,1 vezes a espera média e metade da simultaneidade. As duas são igualmente corretas; a diferença é que a V3 proíbe só o necessário para impedir o ciclo de espera, enquanto a V2 proíbe também situações seguras, como dois filósofos não vizinhos comendo juntos.

**12. Qual é a relação entre o grau de concorrência observado e o tempo total de execução?**

Quanto mais trabalho se sobrepõe, menor o tempo. Sem sobreposição nenhuma (V0) foram 10,2 s. Sobrepondo só o pensar (V2, um comendo por vez), 5,1 s. Sobrepondo também as refeições (V3, dois comendo), 2,7 s. Dobrar o número de filósofos comendo juntos reduziu o tempo quase pela metade. Vale notar que V0 e V2 têm o mesmo pico de simultaneidade (1) e tempos bem diferentes, então o número de filósofos comendo não mede sozinho a concorrência: o pensar ao mesmo tempo também conta.

## 5. Ambiente de execução

| | |
|---|---|
| **Linguagem e versão** | Go 1.27.0 (darwin/arm64) |
| **Sistema operacional** | macOS 13.4 (Darwin 22.5.0) |
| **Processador** | Apple M1 |
| **Cores disponíveis** | 8 |
| **Compilação e execução** | `go run ./v0`, `go run ./v1`, `go run ./v2`, `go run ./v3` (o `go run` compila e executa) |
| **Capturas de tela** | GitHub Codespaces (Linux), mesmo código do repositório. Não foi usado para as medições da tabela. |
| **Experimento completo** | `scripts/rodar.sh` faz 5 execuções de cada versão e grava as saídas em `resultados/saidas/`. |
| **Código-fonte** | https://github.com/eduardocsilveira/FPPD_T1 |